

我把一句 idea 交给 Agent，又让它先写 spec

六个小项目如何拉开 22.17 分

项目 [Grenzlinie/dsh-development-spec-skill](#)

上游样本 DeepSeek Harness commit [47f943859bef60e4160492346772ded9b24f765a](#)

实操范围 React 前端、FastAPI 后端、Canvas 小游戏，共三组配对实验

一句话 人保留产品决定，Agent 按获批 spec 完成实现、验收和记录



1 项目动机

Coding agent 已经能把一句需求迅速变成能跑的代码。速度提高以后，产品里的空白也更容易被它自行补齐。失败状态怎样排序，移动端允许省掉哪些信息，重复请求该成功还是冲突，重启

以后哪些状态必须清零。这些问题没有提前写清，Agent 就会选一个答案继续做。

Research Runboard 的 baseline 首轮漏掉 Attention、失败优先级和 Retry。Seat Hold API 把重复取消继续当成功，状态字段又偏离冻结合同。Signal Drift 把水平中央安全带改成二维移动目标环。三份代码都能运行，修复以后测试也全绿，产品含义却发生了不同程度的漂移。

项目因此选择一条很窄的分工。人回答用户问题、目标、非目标、验收含义和高成本选择。Agent 负责澄清、写 spec、实现、测试、审查与文档同步。每个非平凡变更先建立 Agent Note，产品含义获批以后才进入生产实现。

目标是建立一条可复查的连接

idea → approved decisions → code/tests/docs → evidence

2 从 DeepSeek Harness 学到的结构

DeepSeek Harness 把不同信息放进不同权威层。根 AGENTS.md 保存每次工作都要看到的常驻边界。Architecture 和 testing 文档描述当前系统。Agent Notes 保存仍可重新讨论的决定。测试与 validator 负责拒绝不一致的结果。

仓库历史显示，这套制度经过多次迭代才成形。最初几天先建立 instructions、需求和架构，随后补入 review 修复、决策记录与质量 gate。部分 ADR 是回填，所以本项目没有把今天的规则倒推成上游从第一天就完整执行的流程。

在固定快照上重跑分析，全部 refs 共 12,293 个 commit，non-merge 有 6,683 个。2,028 个 non-merge commit 修改 Agent Notes，1,441 个同时修改 Note 与 production path，1,508 个同时修改 Note 与 tests。这些计数证明文档和代码存在大规模共同演化。它们无法证明某一份 Note 必然先于代码，效果也需要另做实验。



原文 Fig. 1: 本项目从 DeepSeek Harness 的文档分工中抽出的八步开发循环。

本项目保留了五条机制。

- 人与 Agent 的 decision ownership 分开，批准点落在产品含义和高成本选择上。

- 非平凡工作先形成 proposal，验收标准只描述 observable outcome。
- Instructions、current-state docs、decision record 和 tests 各有权威范围。
- Proposed 与 implemented 使用不同语气，完成以后记录已经成立的事实。
- 能机械检查的规则进入真实 gate，交付时保留实际结果。

3 Skill 框架

spec-driven-development 使用八步循环。DISCOVER 先读仓库和约束。FRAME 把 idea 写成问题、目标与边界。SPECIFY 形成 Agent Note。APPROVE 由人确认产品含义。批准以后进入 IMPLEMENT、VERIFY 和 REVIEW，最后用 RECORD 把 proposal 改写成当前决定。

产品含义变化时，工作退回 APPROVE。模块划分、内部接口和测试组织等可逆选择由 Agent 继续处理。这个边界让人离开普通工程微管理，也让 Agent 无法用“技术上更容易”替代产品批准。

Skill 自带标准库实现的 specctl.py。它检查治理文件、创建和校验 Note、限制状态迁移并执行项目登记的验证命令。所有写操作默认先预览，显式加入 --apply 才落盘。一次 diff 是否属于非平凡变更仍由 Agent 做语义判断，工具不会根据文件名假装理解产品影响。

人主要回答五类问题

用户遇到了什么，哪些结果算成功，哪些内容明确不做，哪些选择代价很高，失败以后是否改变原目标。

4 实操检验怎样保持可复查

实验选了三类项目。

项目	技术	主要产品风险
Research Runboard	React 与 TypeScript	状态、排序、移动信息与 Retry
Seat Hold API	FastAPI 与 SQLite	幂等、四态、并发与重启持久性
Signal Drift	Canvas 与 TypeScript	确定性、计时、暂停与玩法边界

每组共享同一份 idea、公开资源、builder model、reasoning effort 和一次标准 repair。Product oracle、分值与 condition map 在 builder 启动前锁定并记录 SHA-256。Treatment 唯一增加的条件，是显式使用本项目 Skill 与治理流程。

同一个 product owner simulator 只回答产品行为、优先级和验收含义。Baseline 可以提问，三组都没有提问。Treatment 分别经过 2、3、3 轮 proposal review 才获批准。Blind evaluator 先只看匿名源码、运行结果和 product oracle，冻结 70 分产品 scorecard，随后才读取过程 artifacts 评 30 分，最后解盲。

六套项目均完成 fresh install，所有测试与构建通过。四个 Web 项目在 1440 × 900 和 390 × 844 下完成真实浏览器检查，共保留 10 张截图。

5 结果怎样算出来

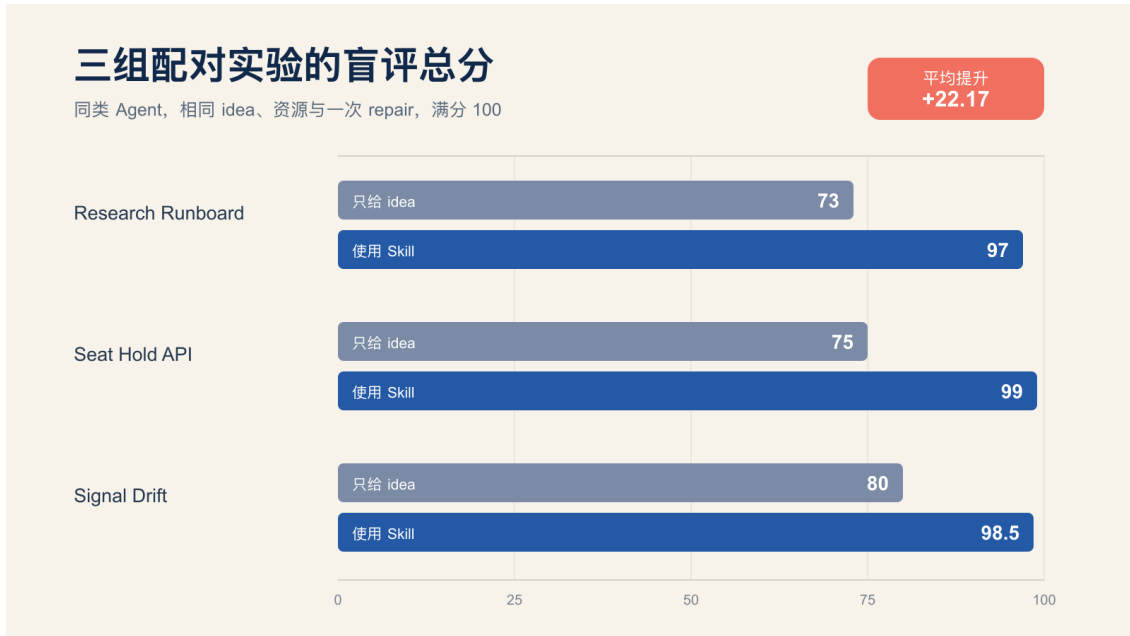
对第 i 个 case，最终总分由产品分与过程分相加。

$$T_i = P_i + R_i, \quad P_i \in [0, 70], \quad R_i \in [0, 30]. \quad (1)$$

P_i 检查隐藏验收、robustness、产品质量与 test quality。 R_i 检查 spec traceability、当前文档一致性和可复现交接。Treatment 相对 baseline 的平均变化写成

$$\begin{aligned} \overline{\Delta T} &= \frac{1}{3} \sum_{i=1}^3 (T_i^{\text{skill}} - T_i^{\text{idea}}) \\ &= \frac{(97 - 73) + (99 - 75) + (98.5 - 80)}{3} \\ &= \frac{24 + 24 + 18.5}{3} = 22.1667. \end{aligned} \quad (2)$$

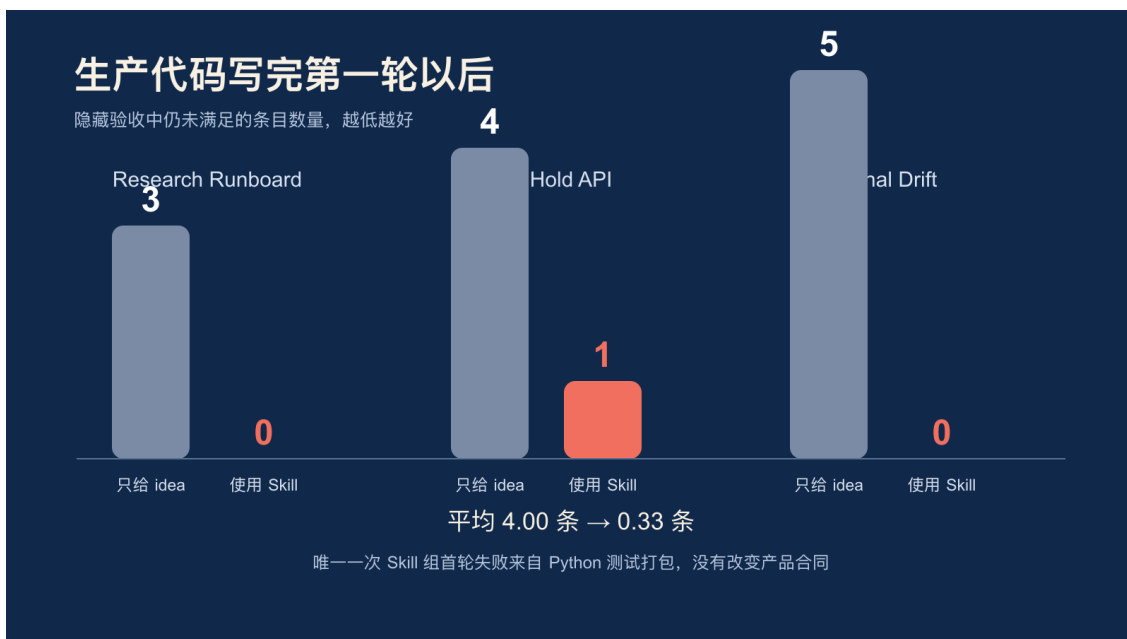
关键就在这一步。22.17 分纳入了三对匿名盲评的全部差值。产品分平均增加 6.67，过程分平均增加 15.50。



原文 Fig. 2: 三组配对实验的匿名盲评总分，蓝色为显式使用 Skill 的 candidate。

首轮实现结束时，baseline 分别留下 3、4、5 个隐藏验收失败项。Treatment 是 0、1、0，唯一一次失败来自 Python 测试 package 的导入方式，没有改变 API 合同。

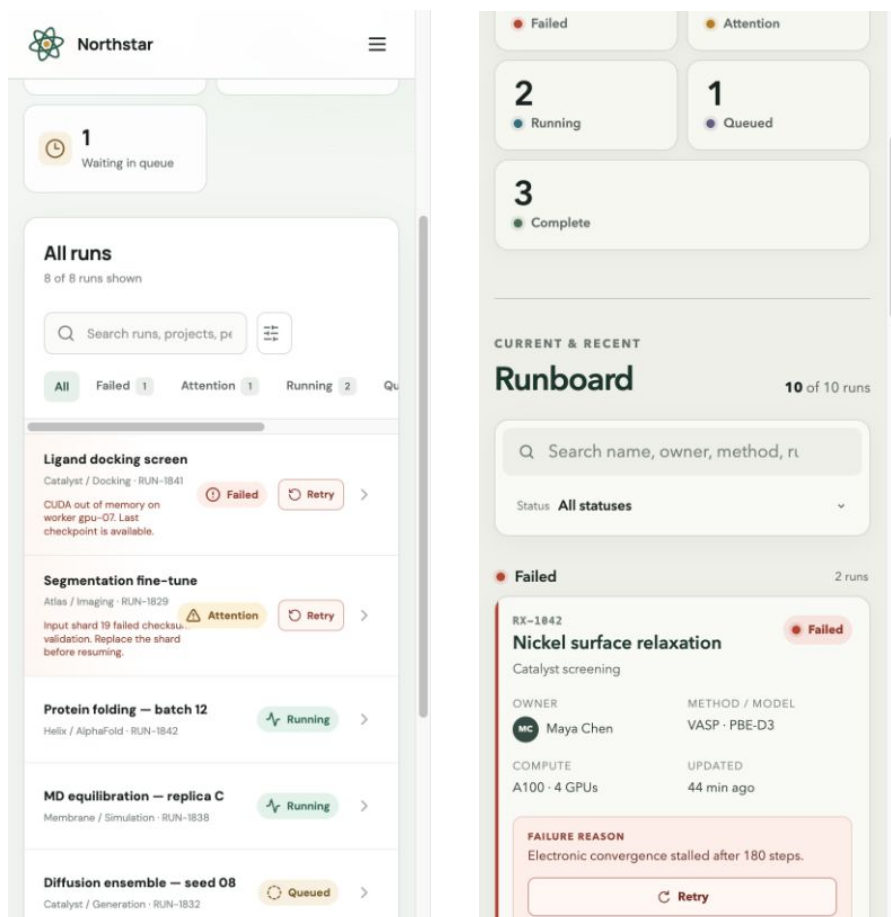
$$\overline{F}_{\text{idea}} = \frac{3 + 4 + 5}{3} = 4.00, \quad \overline{F}_{\text{skill}} = \frac{0 + 1 + 0}{3} = 0.33. \quad (3)$$



原文 Fig. 3: 生产代码首轮完成后仍未满足的隐藏验收条目，数值越低越好。

6 截图里能看见的差别

Runboard 的 baseline 在 repair 后已经可用，390px 页面也没有整页横向溢出。内部列表仍需要横向滚动，owner、environment 和 timing 无法在卡片中直接扫读。Spec 组将这些字段、失败原因和 Retry 留在同一张移动卡片里。

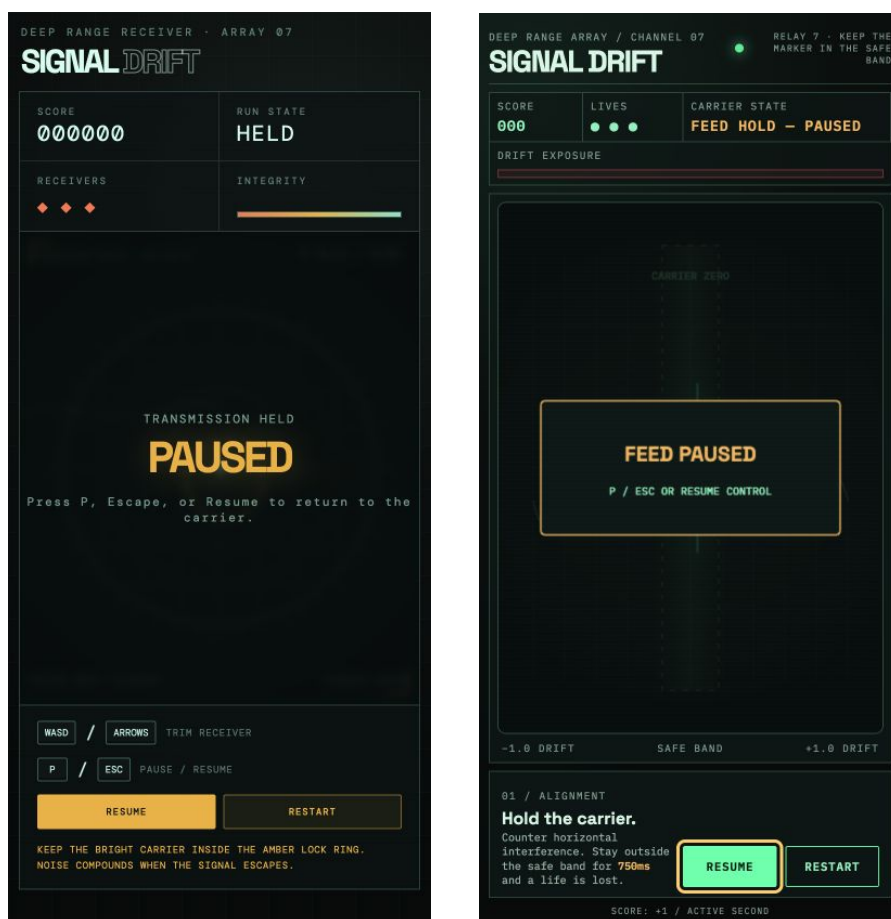


只给 idea

使用 Skill

原文 Fig. 4: Research Runboard 在 390px 移动视口下的最终实机截图。

Signal Drift baseline 的视觉完成度并不低。它保留二维移动目标环与上下控制，已经偏离获批前应明确的水平中央安全带。Spec 组通过 product review 在生产实现前固定 safe band、750ms damage、800ms grace、可见暂停与完整 restart。



只给 idea

使用 Skill

原文 Fig. 5: *Signal Drift* 的暂停状态，左侧玩法目标已经发生偏移。

7 我的评价

7.1 站得住的地方

- 六套候选保存完整 source、lock、tests、first-pass evidence、final evidence 与 handoff，结果可以重复检查。
- 产品分在过程材料打开前冻结，condition map 在评分完成后才解盲，减少了身份偏见影响产品判断。
- Baseline 的三个最终偏差都发生在 tests 全绿以后，说明独立 product review 与 spec 对照确实提供了额外信息。
- Treatment 的限制也被保留。Runboard 的治理命令未直接包含关键移动端测试，Seat API 的错误 envelope 仍不稳定，Signal Drift 的自然 blur 事件保持 unresolved。

7.2 需要打折扣的地方

- 样本量只有三对，builder 也属于同一类模型，22.17 分不能当作其他模型、团队或大系统的预期收益。

- Product owner 与 evaluator 都是模拟角色。它们比 builder 隔离得更好，仍不能代替真实用户研究和专业领域审批。
- 四个 Web 项目完成双视口检查，没有进行完整 screen-reader 审计。两个 API 项目完成 package build，没有进入真实生产部署。
- 文档和 Note 会增加前置成本。Typo、格式化与局部机械修改应当豁免完整生命周期。

7.3 一句总评

在这三个 case 里，显式 spec 与真实批准减少了首轮产品合同漂移。Agent Note、当前文档和 verification evidence 又让 evaluator 更容易发现测试通过以后仍存在的偏差。这个结果适合支持一次小范围采用，尚不足以支持普遍因果判断。

怎样开始 选一个一周内能做完、产品含义又有歧义的 feature。记录批准前发现的歧义、first-pass failure 和独立 review 仍发现的偏差。流程若只增加文档而没有减少这些量，就继续缩短模板和批准面。

完整材料 [GitHub 仓库](#)保存 Skill、采用指南、六套候选源码、冻结 oracle、blind scorecard、命令结果与 10 张截图。